

Threat-Priority Scheduling — Implementation Brief

For the RL lane. Rewritten 2026-09-11 after the §2 gate was run. One change to the scheduler: it accepts a priority input and learns to use it. Nothing frozen moves.

Owner: Aamir. **Decision:** **D70**. **PDF:** `python -m scripts.md2pdf docs/project/THREAT_WEIGHTING_BRIEF.md`.

Outranked by SIH26055_PROBLEM_STATEMENT.md, DECISIONS.md, ENVIRONMENT_SPEC.md, EVALUATION.md.
Where this and those disagree, those win.

0. Status: the gate passed, and it changed the design

The two tests that could have killed this ran on 2026-09-11 against all 47 train configs. **The idea survives. The version in the previous draft does not.**

	Measured	Consequence
HIGH share of detectable emitters	653 / 1704 = 38.3%	Matches the 38.5% previously recorded (within 3 emitters).
Bands holding both a HIGH and a LOW emitter	29.1% of 772 occupied bands	The stated kill condition ("most") does not fire .
Bands where p = HIGH under max over contributors	77.8%	This is the real failure. Median config: 14 of 18 occupied bands. p is near-uniform on the bands that matter.
Band priority as an emitter classifier (max rule)	precision 0.452 vs base rate 0.383	A 7 pp lift. Not worth a retrain.
Same, with share-weighted priority	p 0.92–0.97 vs 0.54; threat-density lift 2.22× at 10–25% airtime, 1.69× at 50%, against a flat 1.17× for the max rule	This is the design.

So: p is a graded, share-weighted quantity per band. Not a 3-bucket max. That was listed as an optional refinement in the previous draft; it is now the specification.

Quote the lift with its airtime fraction. The 2.22× is partly arithmetic: 271 of 772 bands hold *only* HIGH emitters, so any top-25% cut is entirely pure-HIGH and scores 1.000 by construction. The claim that survives cleanly is **a graded priority can rank bands and a three-bucket one cannot** — 334 distinct values against 3, and a flat 1.17× for the max rule at every airtime fraction because 77.8% of bands tie at its top value.

Why it matters: the scheduler has a measured threat bias, and it grows

Scored on the 47 stare replays × 3 seeds, censored intercept time split by threat class (5,112 emitter observations per scheduler). **This is brief step 2 — now done.**

Scheduler	HIGH	MEDIUM	LOW	HIGH penalty
Round-robin (floor)	3.94 s · cov 0.88	4.12 s · cov 0.88	3.96 s · cov 0.88	−0.01 s
Recency (the bar)	2.84 s · cov 0.92	2.93 s · cov 0.91	2.47 s · cov 0.94	+0.37 s
NARADA RL (rung 17c)	2.53 s · cov 0.92	2.34 s · cov 0.94	1.86 s · cov 0.96	+0.66 s

Blind round-robin is fair; every adaptive scheduler is not, and ours is the worst offender. It reaches HIGH-threat emitters 0.66 s later than LOW ones and misses 4 pp more of them entirely. Mechanism, from two earlier measurements: adaptive schedulers chase activity, and threat here runs *inverse* to activity — threat fraction by band-density quartile 0.785 / 0.574 / 0.459 / 0.350 sparsest-to-busiest, HIGH-class median power 12 kW against MEDIUM's 20 kW. Quiet, sparse, dangerous.

This is the justification for the whole feature, and it is our own result, not an argument. It also sets the target: the retrain must shrink that penalty without losing the D71 headline. *Caveat: it uses the §5 example threat library, which is our judgement on 68 radar names.*

1. What we claim, and what we do not

We do not define threat and we do not infer it. Threat is doctrine plus mission context. Measured, the receiver cannot recover it: across 772 occupied band-pairs, against the narrow weapon-directing class, **every band-level observable is within 0.060 of a coin flip** (max level 0.485, mean level 0.497, hit rate 0.560, intermittency 0.472). So priority arrives from outside. That is forced by measurement, not chosen for convenience.

State the scope precisely. The mechanism is $S = \max$ over contributors plus one declaration per cell — which receiver.py itself labels a modelling choice with a per-pulse version deferred, and D28 calls "*an implementation choice taken inside this design (routine, not gated).*" Say "**in this architecture**", never "*at the scan-scheduler layer*" as a general truth. A domain reviewer will know that a real receiver sees the quiet pulse; what it cannot do without deinterleaving (out of scope, D12/D19) is *attribute* it.

The interface is not novel, and we say so first. Two papers already in docs/reference/scheduling/ have it:

- **Köksal** (optimumsearch.pdf, ch. 6, pp. 85/92/106) — "*assign different priority to each frequency band*", operator-set, with the tradeoff named. **Per band, exactly our object.**
- **Dutertre**, RTSS'02 (Dynamic Scan Scheduling.pdf, pp. 1–2, 8) — per-emitter-type weights that "*vary during a mission*", entering as "*a reward received whenever an emitter of type e is detected*"; a uniform-weight control arm; and a scorecard split Critical / Noncritical.

D70's sentence "no scan-scheduling paper in the reference set has it" is **false**, and names Köksal among the exonerated. It needs correcting. **What is genuinely ours:** both papers assume *disjoint* bands and an a-priori emitter table; our bands overlap by half (D3) and our PS says "*in the absence of prior reliable intelligence.*" Nobody asked whether the band is a valid unit for priority under overlap. **The measurement in §0 is the contribution — not the feature.**

2. What gets built

The scheduler takes a per-band priority vector $p \in \mathbb{R}^3$.

p = 1 everywhere -> the plain PS objective. The default. Nothing changes.
p graded -> the scheduler re-plans around what matters this mission.

How p is computed (from a threat library, offline):

$p[b] = \sum_e w(\text{class of } e) \cdot \text{share}_e(b)$ over emitters e known to occupy band b
where $\text{share}_e(b) = e\text{'s illuminations in band } b / \text{all illuminations in band } b$
and $w = \{\text{HIGH } 1.0, \text{ MEDIUM } 0.5, \text{ LOW } 0.0\}$, then rescaled so 1.0 = "ordinary"

Three things this fixes that the max rule did not: 334 distinct values instead of 3; it degrades gracefully as band mixing rises; and it satisfies D55 (a block pinned at its ceiling on 77.8% of bands is D55's complaint with the sign flipped).

How it enters:

- 1. **Observation: 183 → 219.** (D67 took the base vector to 183; the previous draft's "146 → 182" is stale.) Scale so p = 1.0 means ordinary, per D55.
- 2. **Sampled per episode** during training so the policy learns to use p rather than memorise one setting. The sampling distribution is **open** — fix it before the retrain (§4).
- 3. **Reward: a new term, then a re-screen.** Weight **discovery credit only**, never per-slot occupancy — a per-slot threat term pays repeatedly for camping, which is D53's failure mode with a new coefficient on it. newly fires once per (emitter, band) pair (D51), so it is bounded and cannot.

Read this before writing the reward. reward_balance — the only D62-passing candidate and the current DEFAULT_REWARD — **does not read newly at all** (D53 records this; only reward_explore consumes it). So this is *adding a term*, not multiplying an existing one, and **the result must be re-run through python -m rfenv.reward_gate**. Measured: $a + 2.0 \cdot p[b] \cdot \text{len}(\text{newly})$ term is **33–59% of total episode reward** — large enough to disturb the rung ordering D62 exists to protect. Start at a coefficient near 0.5 and screen upward.

3. Order of work

	Do	Cost	Kill condition
0	~~P2 band mixing~~ · ~~P1 split coarseness~~	—	DONE 2026-09-11. Passed, with the design change in §0.
1	The inference-time knob. Reweight the sampled action distribution by <code>log p</code> at inference (D54 already samples). Works on the frozen 17c checkpoint — no retrain, no observation change.	Hours.	— This is the demo, and the fallback if step 3 fails.
2	Implement graded p (§2) as code, plus the 68-name threat library, before any scheduler is scored against it.	A day.	—

	Do	Cost	Kill condition
3	Add p to the observation, add the reward term, re-run reward_gate , retrain — with the p = uniform control arm alongside .	The retrain. Every checkpoint dies, as at D49/D55/D67.	Screen fails → re-tune the coefficient, do not ship.
4	Ablations. Two arms, not one. (a) permutation: shuffle p , re-run; (b) correct-p : the true graded vector.	Minutes.	(a) unchanged behaviour → feature dead.
5	Select with <code>rfenv/selection.py</code> (D61), score on the 12 validation configs (D60).	Existing machinery.	—
6	Report <code>EVALUATION.md</code> §4 split by threat class, alongside the pooled table — never instead of it.	—	HIGH intercepts per episode too few → report pooled with effect size.

Step 1 before step 3. It de-risks the demo completely and costs a morning.

4. Open — decide before the retrain

1. **Share weight:** pulse-share or cell-share. Both measure $2.22\times$. **Recommend pulse-share** — illuminations are the interception-ratio denominator. Fix in code first.
2. **p on empty bands.** ~19 of 36 bands are unoccupied in a typical config and p there is undefined. That is a third of the vector.
3. **The p sampling distribution** during training (D70's own open item, still open).
4. **In-episode p update — recommend DEFER.** It needs emitter *identity*, which needs deinterleaving, which D12/D19 put out of scope. So it is not a stand-in for something we abstracted away; it is a stand-in for something we **excluded**, feeding the observation a value no component of our architecture can produce. Episode-start p raises no such question. If the in-episode story is wanted later, the clean form reads only Y and stays inside D29.

5. Known problems

#	Problem	Status
P1	HIGH is 38.3% of emitters — p near-uniform.	Measured. Narrowing to fire-control-only gives 15.8% of emitters but 53.4% of bands — narrowing works on emitters, not on bands. Graded p is the fix, not a narrower class.

#	Problem	Status
P2	Priority is per band; threat is per emitter.	Measured and answered by §2. 1.15× → 2.22×.
P3	Thin scoring — too few HIGH intercepts per episode to split the table.	Open. Count it under rung 5 before committing to a split table.
P4	The agent ignores p .	Open. Needs both ablation arms (§3 step 4): a random- p permutation passes for the wrong reason.
P5	No control arm.	Non-negotiable. Train $p = \text{uniform}$ alongside, same split, same seeds.
P6	p needs scaling.	Answered — graded p with 1.0 = ordinary (D55).

Deleted from the previous draft, with reasons:

- *Split HIGH by search vs lock-on* — **impossible on this data.** `scan_config.attrs['scan_type']` takes exactly two values across all 2,363 transmitters: `Circular` (2,258) and `Omni` (105). No track, no lock, no sector. Situational threat is **unrepresentable** here, not deferred.
- *Cite external doctrine for the ratings* — **no obtainable source.** Per-radar threat ratings live in classified national libraries; open doctrine describes prioritisation without publishing ratings. Instead: publish the mapping as code with one line of rationale per name, and report the narrow fire-control class as a **sensitivity arm**.

6. What does not move

`rfenv/constants.py` untouched. No gate re-runs, no baseline re-runs. `DEFAULT_REWARD` stays `reward_balance` as the control arm. D20's cold start is unaffected: the *scheduler* still starts with no emitter intelligence — p is mission input, not a learned prior.